



SE618 LOGISTIC CASE STUDY ANALYSIS

WEEK 7 – PROGRESS REPORT

SERGE – AI Agent for Supply Chain Operations

Gauthier SCHMITT	6822807466
Louis GRIGNOLA	6822807516
Matteo SCHWARZ	6822807557
Maël TELLUS	6822807532

SUBMITTED TO:
AJARN WARUT

MARCH 2026
SEMESTER 2/2025

**SCHOOL OF MANUFACTURING SYSTEMS AND MECHANICAL
ENGINEERING**
SIRINDHORN INTERNATIONAL INSTITUTE OF TECHNOLOGY
THAMMASAT UNIVERSITY

CHAPTER 1

WEEK 7 SUMMARY

This report covers the work completed during Week 7 (March 16–19, 2026). The focus this week was on **completing the MCP server implementation, validating it with live agent queries, and creating project documentation.**

1.1 Objectives for This Week

1. Complete the Go MCP server with all 21 tools across the 4 supply chain systems
2. Validate the agent can perform cross-system reasoning on the three use cases
3. Document the MCP setup process for reproducibility
4. Document the supply chain modelisation and roadmap

1.2 Status: All Objectives Met

All four objectives were achieved. The MCP server is fully functional, the agent successfully demonstrated all three use cases (order status, root cause analysis, supplier comparison), and documentation was produced.

CHAPTER 2

MCP SERVER IMPLEMENTATION

2.1 Architecture

The MCP server is implemented as a **single Go binary** that loads seed data from JSON files at startup and exposes 21 read-only tools via the Model Context Protocol over stdio. The architecture follows a clean layered design:

- **Repositories:** In-memory data stores loaded from JSON seed files
- **Services:** Business logic (e.g., computing supplier on-time rates from PO history)
- **Controllers:** Interface between services and the MCP layer
- **MCP Server:** Tool registration and stdio transport via `trpc-mcp-go`

The server is built with Go 1.23 and depends on `trpc-mcp-go` v0.0.12 for the MCP protocol implementation.

2.2 Tools Implemented

A total of **21 MCP tools** were implemented across the four supply chain systems:

Table 2.1: MCP tools by system.

System	Tools	Key Capabilities
SRM (Procurement)	7	Supplier lookup, PO filtering by status/supplier, overdue POs, supplier comparison
WMS (Warehouse)	6	Product lookup by ID/SKU, inventory check, low stock detection, stock movements
OMS (Orders)	5	Order details, customer lookup, orders by customer/status, overdue orders
TMS (Transport)	4	Shipment details, order tracking, exception shipments, carrier info
Total	21	

2.3 Seed Data

The MCP server loads simulated data representing a **Thai B2B mango distribution company**. The data is stored in 11 JSON files:

Table 2.2: Seed data overview.

Data File	Records	Description
suppliers.json	3	Thai mango farms (Chanthaburi, Chiang Mai, Prachuap)
purchase_orders.json	~10	Procurement orders to suppliers
products.json	5	Mango product SKUs (premium, standard, assorted)
warehouses.json	1	Bangkok central warehouse (WH-BKK)
inventory.json	5	Stock levels per product
stock_movements.json	~20	Inbound/outbound movement records
customers.json	50	Thai B2B customers (restaurants, hotels, supermarkets)
orders.json	500	Customer sales orders
carriers.json	2	Kerry Express (ground) + SCG Cold Chain (refrigerated)
shipments.json	400	Delivery shipments
tracking_events.json	~1000	Shipment tracking history

CHAPTER 3

USE CASE VALIDATION

The three use cases defined in Week 4 were tested with the live MCP server connected to Claude Code. All three were validated successfully.

3.1 UC-1: Order Status Inquiry

Query: “What is the status of order ORD-005?”

The agent correctly:

1. Called `oms_get_order` to retrieve order details (customer, status, dates, line items)
2. Called `tms_track_order` to find the associated shipment and tracking info
3. Returned a unified response with order info + shipping status

Result: The agent identified that ORD-005 was shipped on Feb 10, two days after being placed and four days before the required date. It correctly listed both line items with quantities and prices, and computed the total (\$14,750).

Tool calls: 2 (optimal)

3.2 UC-2: Delayed Order Root Cause Analysis

Query: “Why is order ORD-007 delayed?”

The agent correctly performed the full cross-system investigation:

1. Called `oms_get_order`: order is processing, priority urgent, required by Feb 12
2. Called `tms_track_order`: no shipment found (order stuck at fulfillment)
3. Called `wms_check_inventory`: only 8 units available, 50 needed (shortfall of 42)
4. Called `wms_get_product`: confirmed product is Nam Doc Mai Mango Box 5kg

Result: The agent correctly identified the root cause: *insufficient inventory* (8 units available vs. 50 required). It noted the order is 5+ weeks overdue and the inventory is below the reorder point of 20.

Tool calls: 4 (appropriate for the diagnostic chain)

3.3 UC-3: Supplier Performance Comparison

Query: “Compare the performance of our suppliers”

The agent correctly:

1. Called `srm_compare_suppliers` to get side-by-side performance metrics
2. Called `srm_list_suppliers` to get full supplier profiles
3. Generated a structured comparison table with all key metrics

Result: The agent produced a detailed comparison identifying Somchai Mango Farm as the top performer (100% on-time, 2.7-day avg lead time) and Northern Fruits Co. as the weakest (0% on-time, 8.5-day avg lead time). It included actionable insights about which supplier to use for urgent orders.

Tool calls: 2 (optimal)

3.4 Validation Summary

Table 3.1: Use case validation results.

UC	Name	Tool Calls	Correct?	Notes
1	Order Status Inquiry	2	Yes	Optimal tool selection, complete data
2	Root Cause Analysis	4	Yes	Full causal chain identified
3	Supplier Comparison	2	Yes	Structured output with actionable insights

These results are preliminary (single runs, not the full 45-run evaluation protocol), but they confirm that the MCP server and agent integration work correctly for all three use cases.

CHAPTER 4

DOCUMENTATION

Two documentation files were created in the `serge/docs/` directory:

4.1 MCP Setup Guide

File: `docs/MCP_SETUP.md`

This guide documents how to build and configure the MCP server from scratch. It covers:

- Prerequisites (Go 1.23+, Claude Code CLI)
- Build command (`go build`)
- Configuration of `.mcp.json` and `.claude/settings.local.json`
- Verification steps
- Full reference of all 21 MCP tools
- Seed data summary

All machine-specific paths have been replaced with `<PROJECT_ROOT>` placeholders so that any team member or future user can set up the system on their own machine.

4.2 Supply Chain Modelisation

File: `docs/SUPPLY_CHAIN_MODEL.md`

This document describes the V1 supply chain model and the roadmap toward a full B2B simulation:

- **V1 (current):** Static, read-only Thai mango distribution – 5 SKUs, 3 suppliers, 1 warehouse, 50 customers, 2 carriers. Validates agent architecture and builds evaluation benchmarks.
- **V2:** Dynamic operations – write operations, time simulation, multi-warehouse, more product categories.
- **V3:** Realistic B2B – demand forecasting, contracts, payment terms, quality management, multi-carrier optimization.
- **V4:** Full simulation company – international trade, real-time events, financial integration, compliance, multi-tenant.

CHAPTER 5

NEXT STEPS

5.1 Week 8 Plan

Table 5.1: Week 8 planned tasks.

Task	Description
Ground truth definition	Define the 15 evaluation scenarios (5 per UC) with expected answers from seed data
LLM-as-Judge pipeline	Implement the judge evaluation script with the 5-criterion scoring rubric
Full evaluation runs	Execute 45 runs (15 scenarios $\times$ 3 runs each) and collect results
Results analysis	Compute aggregate scores, consistency metrics, and tool call efficiency
Final report	Begin writing the final report with evaluation results